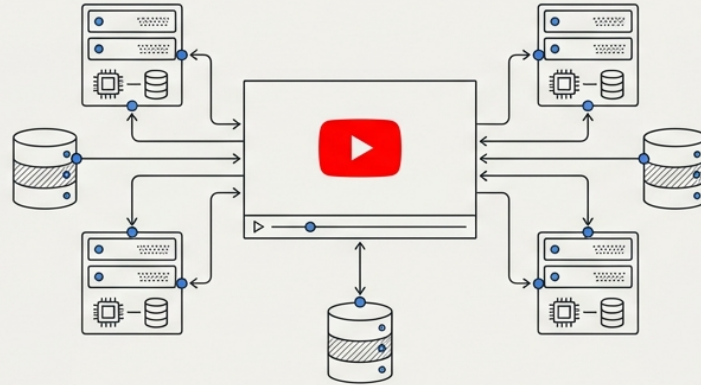


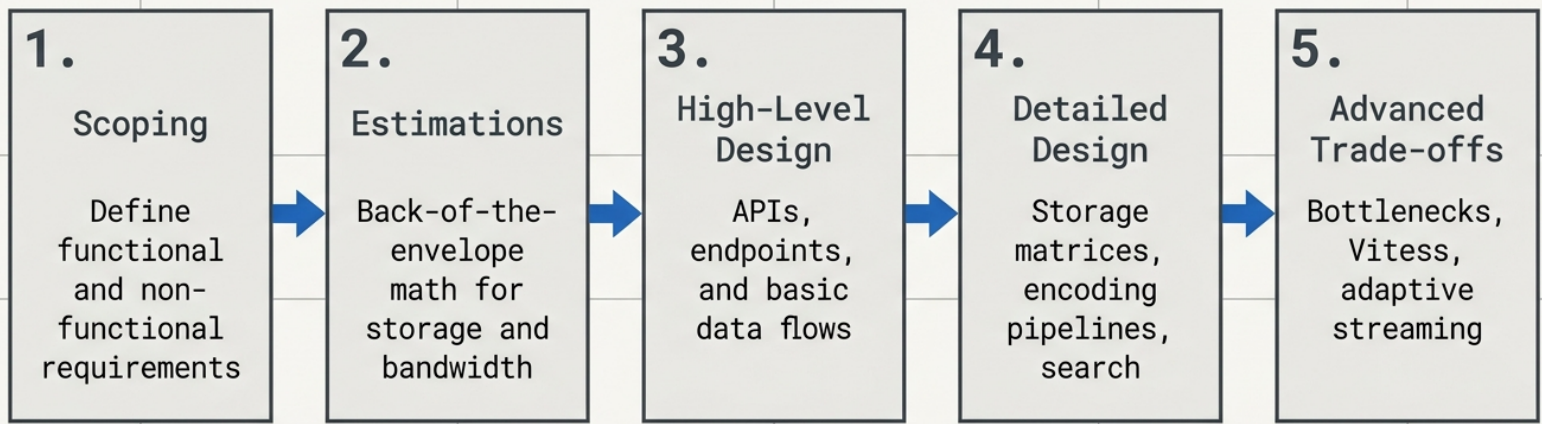
Cracking the YouTube Architecture

System Design Interview Playbook

A structured playbook for scaling a video platform to 2.5 billion users.



```
user@interview:~$ ./start_system_design.sh
```



Interview Focus: The interviewer evaluates your structured thinking. Stick rigidly to this framework to control the 45-minute clock.

Functional Requirements (What the system does)	Non-Functional Requirements (How the system behaves)
 Stream videos	High Availability (99%+ uptime)
 Upload videos	Scalability
 Search by title	Low Latency streaming
 View thumbnails	Data Reliability (no lost uploads)
 React (Like/Comment)	

Senior Signal

Explicitly state that strict consistency is NOT required for video distribution. Eventual consistency is the right trade-off—a new upload doesn't need to propagate to all 2.5 billion users instantly.

KEY METRICS

**1.5
Billion**

Total Users

**500
Million**

Daily Active Users

694,000 hrs

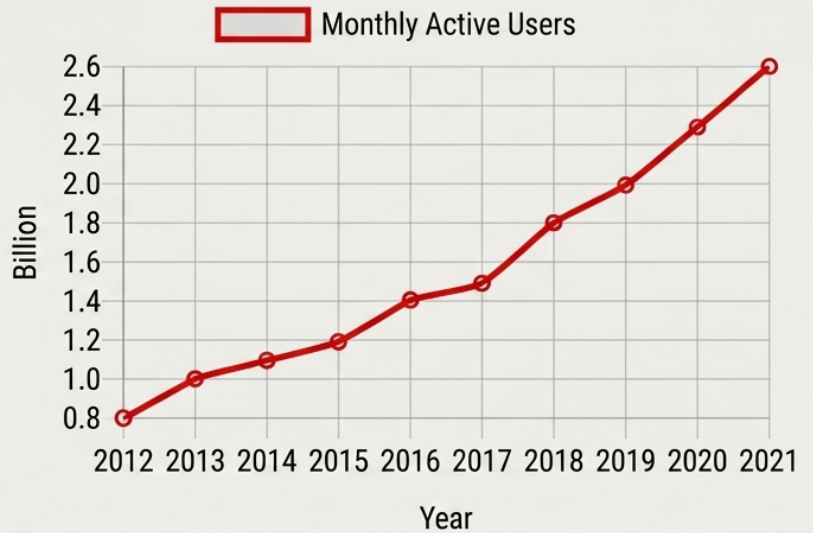
Streamed per minute

500 hrs

Uploaded per minute

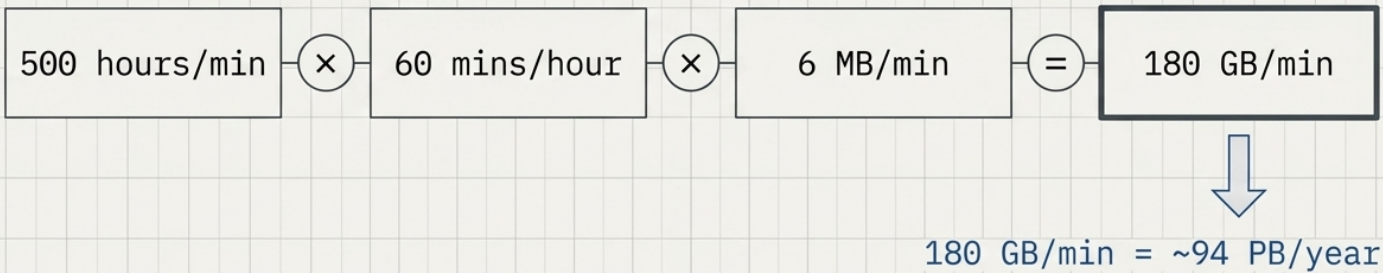
Upload-to-View Ratio: 1:300

YouTube's Yearly User Growth



Interview Focus: Always state your assumptions out loud before calculating constraints.

Storage Math: Visual Equation



Interview Trap

Do not forget transcoding! You must multiply your baseline storage estimate by at least 5x to account for the different resolutions and codecs generated by the encoder.

BANDWIDTH CALCULATIONS: UPLOAD VS. STREAMING

UPLOAD BANDWIDTH ESTIMATION

500 hours/min

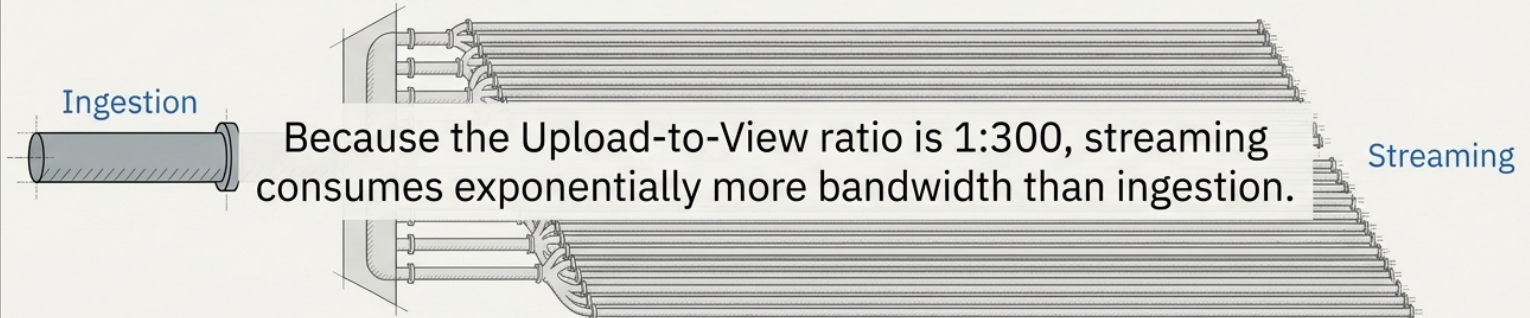


120 MB/min
transmission

=

**480 Gbps
Upload Bandwidth**

READ BANDWIDTH MULTIPLIER (1:300 RATIO)



Assuming 500M DAU, peak load requires approximately ~8K servers based on RPS calculations.

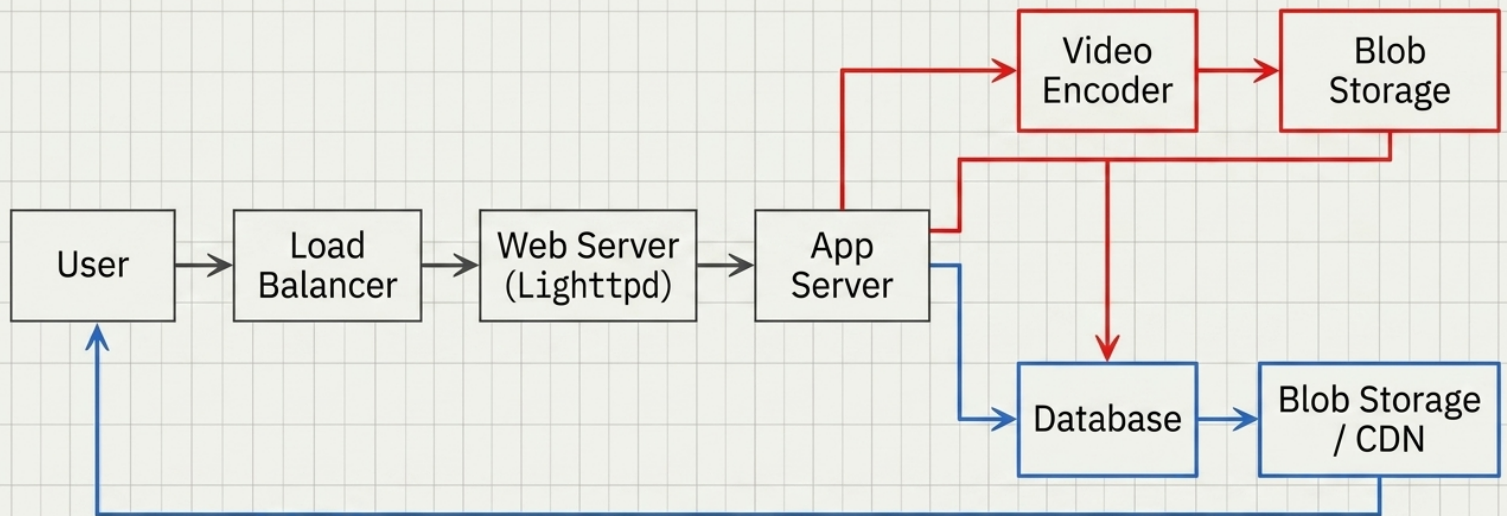
API Documentation

POST /upload	user_id, video_file, default_language, privacy_settings
GET /stream	video_id, screen_resolution, user_bitrate, device_chipset
GET /search	search_string, length, quality

Senior Signal

Including user_bitrate and screen_resolution in the /stream endpoint proves you are already anticipating Adaptive Bitrate Streaming before you even draw the architecture.

SYSTEM ARCHITECTURE: WRITE & READ SEPARATION



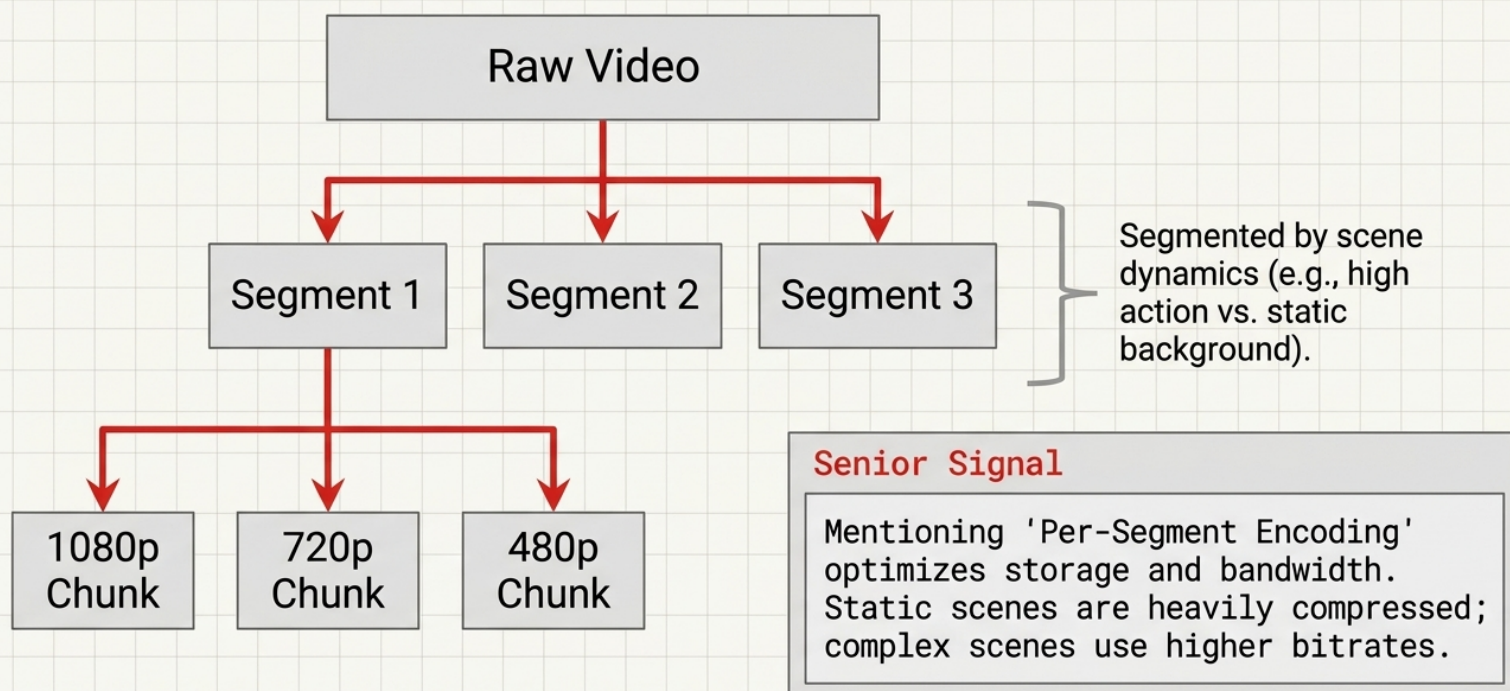
Nail the separation of concerns. Never pass massive video blobs directly through your business logic application servers.

Storage Choice Matrix

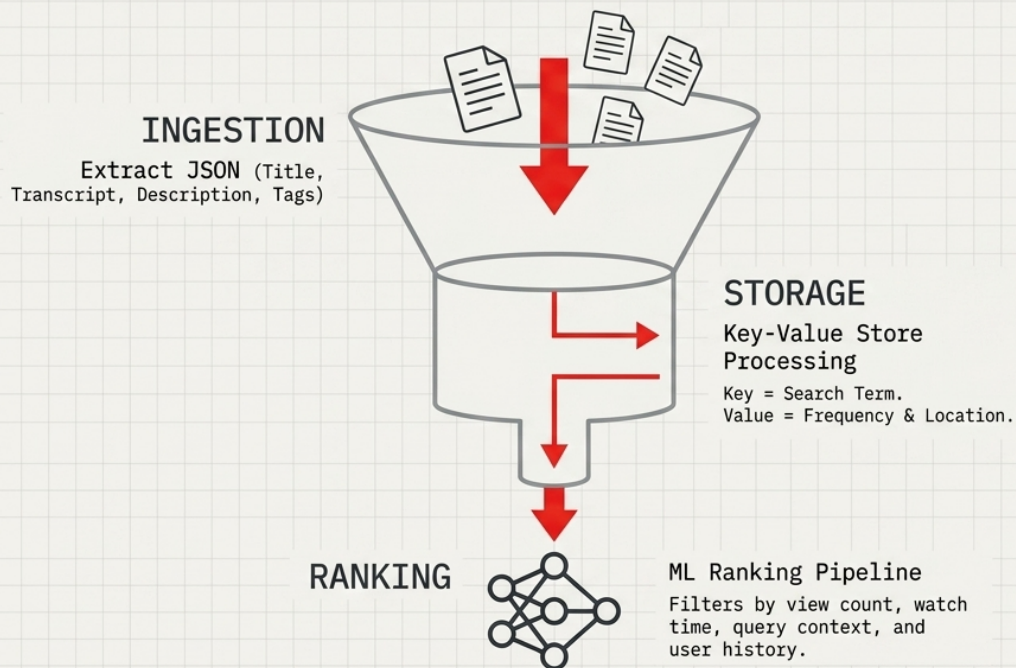
User & Metadata	Thumbnails	Video Files
MySQL (Relational)	Bigtable (NoSQL)	Blob Storage (GFS/S3)
Requires structure, ACID properties, and efficient query searching.	Massive volume of small key-value data requiring high throughput and low-latency access for long-tail reads.	Unstructured object storage designed for massive files.

A single database technology cannot handle YouTube's scale. Match the datastore to the data shape.

ENCODING PIPELINE: PER-SEGMENT & ABR

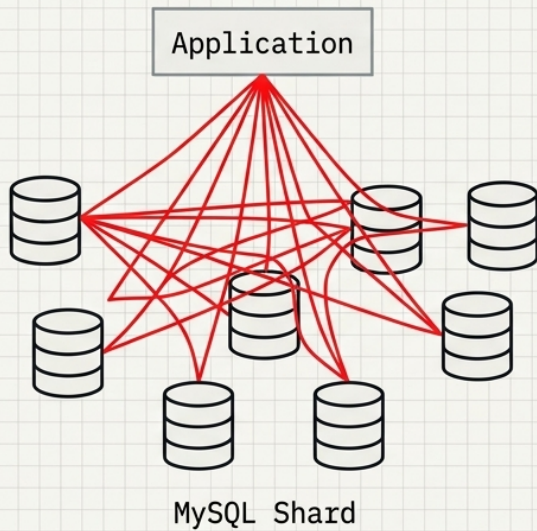


SEARCH INFRASTRUCTURE PIPELINE



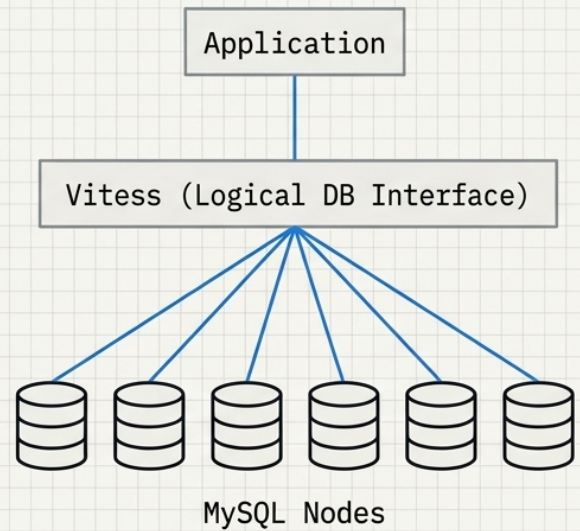
Production search relies on more than keyword matching. The ranking pipeline is required to filter millions of candidates down to dozens.

Standard Sharding



Client manages complex routing logic; threatens ACID compliance.

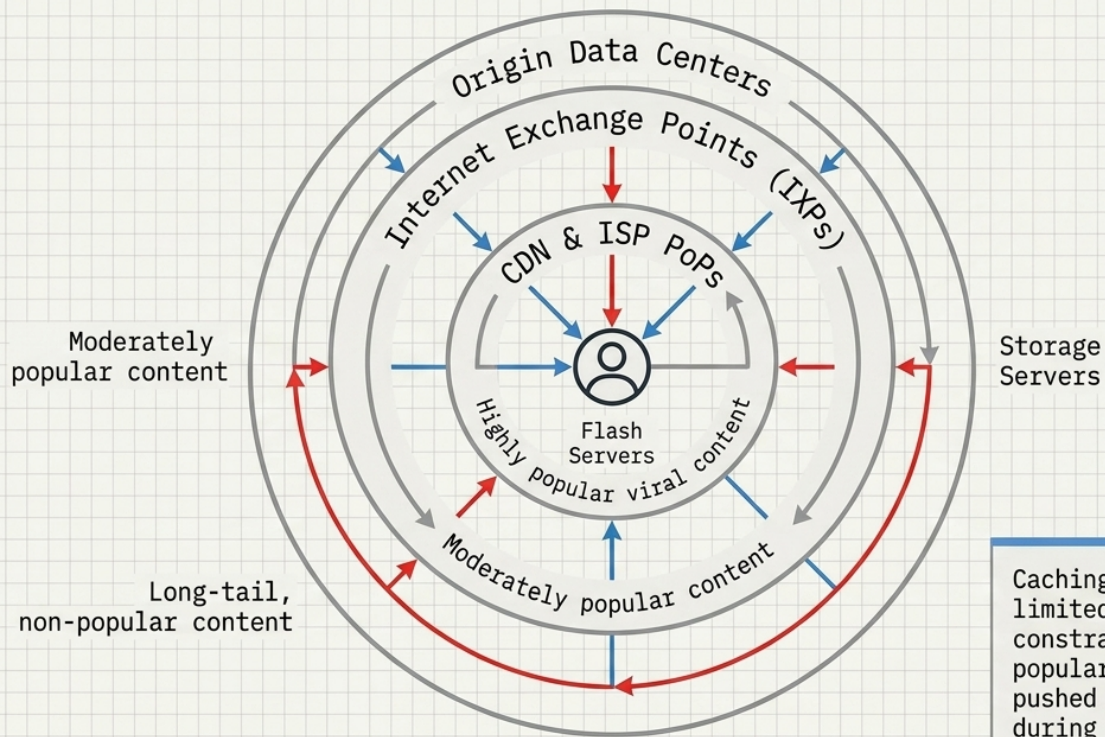
Vitess Architecture



Vitess cleanly routes queries to a massive cluster.

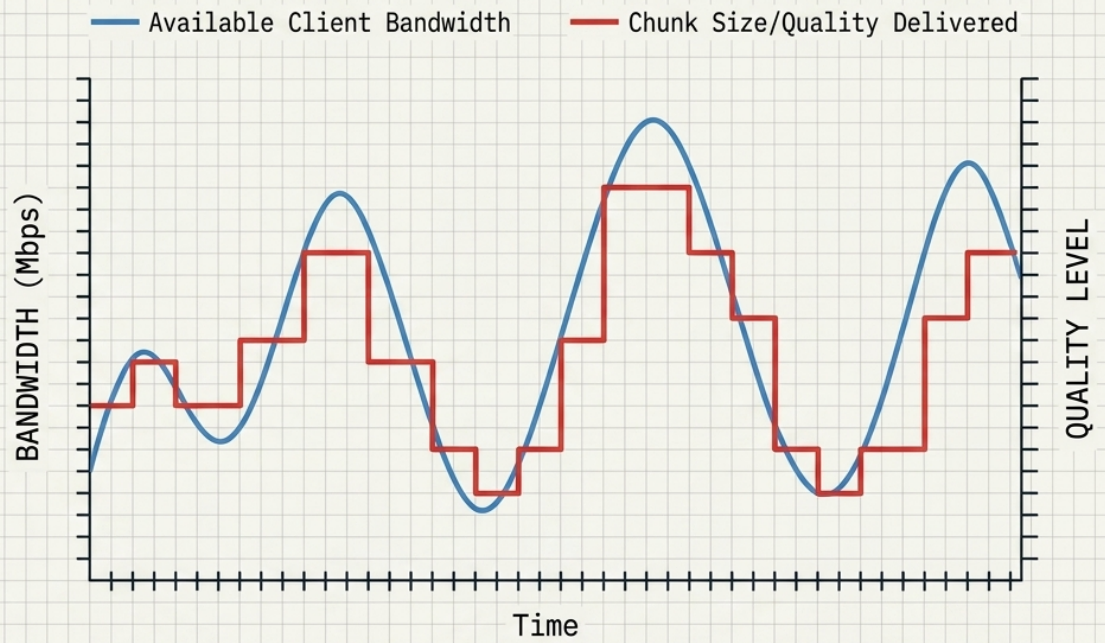
Mentioning Vitess proves you understand the operational nightmare of managing sharded relational databases. It abstracts sharding behind a logical single-database interface.

GLOBAL CONTENT DELIVERY NETWORK HIERARCHY



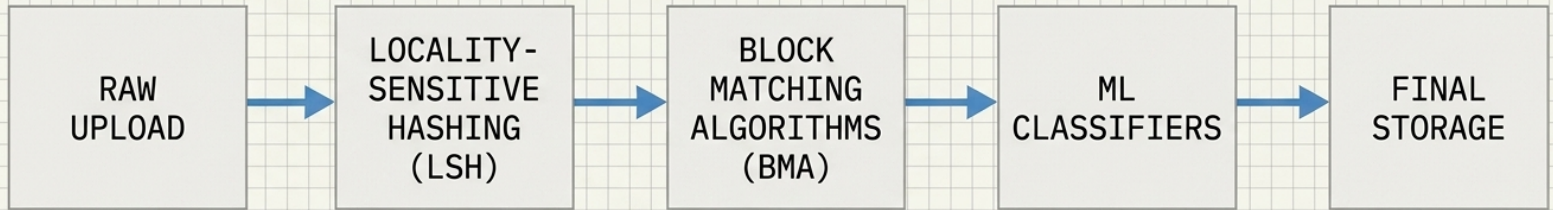
ALGORITHM INPUTS

1. End-to-end available bandwidth.
2. User device capabilities.
3. Encoding techniques.
4. Client buffer space.



The client continuously measures bandwidth and requests the appropriate chunk quality dynamically.

DEDUPLICATION DATA FLOW & IMPACT ANALYSIS



9.5 PETABYTES SAVED ANNUALLY

Assuming 50 out of 500 hours uploaded are duplicates, deduplication prevents runaway storage costs and ensures copyright compliance.

FAULT TOLERANCE GRID

FAILURE COMPONENT	SYSTEM MITIGATION
Node/Server crash	Heartbeat protocol monitors health; consistent hashing seamlessly removes the dead node.
Database outage	Data partitioning isolates failures; replicated data stores enable immediate re-spawning.
Load Balancer failure	Deploy Active-Passive pairs and use DNS round-robin routing.

System availability is dictated by how elegantly your architecture handles inevitable hardware failures.

- ✓ Trade strict consistency for high availability and eventual consistency.
- ✓ Separate structural metadata (MySQL/Vitess) from massive unstructured blobs (Object Store).
- ✓ Utilize Per-Segment Encoding to enable Adaptive Bitrate Streaming.
- ✓ Push popular content to edge CDNs/ISPs; keep long-tail content in origin data centers.
- ✓ Implement deduplication (LSH/ML) to save petabytes of storage.

Drive the conversation, state your assumptions, and own the whiteboard.